ZTracker — Quick User Guide

Fiji / ImageJ plugin · a two-part pipeline: Z-Projection maker → 3D Z-Coordinate Extractor

What this plugin does

ZTracker turns your **2D cell tracks** into **3D tracks** by adding a Z (depth) coordinate to every detection. It works in two parts, each its own item under **Plugins > ZTracker**, run in order:

1. **Z-Projection + Origin Map** (Part 1) — from a raw Z-stack, it builds the depth-coded projection images and a `.json` depth map.
2. **3D Z-Coordinate Extractor** (Part 2) — it reads those, looks up the depth under each tracked point, and saves 3D tracks.

If a colleague or an earlier run already gave you the depth-coded images and the `.json` map, you can skip Part 1 and go straight to Part 2.

> **Two flavours of Part 2.** Which extractor you use depends on what your depth images hold:
>
> - **Indexed images + a JSON map** (what Part 1 makes) → use **3D Z-Coordinate Extractor**. Each pixel is a *code* that looks up a real depth in the `.json` file.
> - **Direct-Z images, no JSON** (e.g. Fiji's **TopoJ** output) → use **3D Z-Extractor (TopoJ / direct-Z)**. Each pixel value *is* the depth in µm, so there's no map to load and no JSON to pick.
>
> Both share every other step (frame offset, sampling, export). Pick the one that matches your images.

Part 1 — Z-Projection + Origin Map

Makes the two inputs Part 2 needs, starting from a raw Z-stack. Open it via **Plugins > ZTracker > Z-Projection + Origin Map**.

What it needs: a **dataset folder** holding your raw Z-stack, in either of two layouts — you say which with the **Input type** dropdown.

Image depth: your raw slices can be **8-, 16-, or 32-bit** grayscale. Part 1 only compares pixel brightness between depths, so the bit depth affects precision, not whether it works — use whatever your microscope produced. **Colour (RGB) images won't work**: they aren't rejected, but the projection would run on packed colour values and give meaningless depths, so convert to grayscale first (**Image > Type > 8-bit** or **16-bit**).

Input type = ***Z-layer sub-folders*** (the default): sub-folders named by their depth in µm (e.g. `-300`, `-299`, …), each holding one TIFF per time frame (same filename across depths):

```
dataset/
├── -300/   frame_0001.tif, frame_0002.tif, ...
├── -299/   frame_0001.tif, ...
└── ...
```

Input type = ***TIFF stacks***: one multi-slice TIFF **per time frame**, sitting straight in the folder — the slices inside each file are the depths:

```
dataset/
├── 00001.tif     ← time frame 1, one slice per depth
├── 00002.tif     ← time frame 2
└── ...
```

Any numbering works (`00001.tif` , `7.tif` , `img_0010.tif`) — files are ordered by the number they **end with**. With this layout the depths are read from each slice's label inside the file (the `z = -400.000` labels Fiji writes for a Z-stack), since there are no depth-named folders to read them from. If a slice's label has no depth in it, the tool stops and tells you which slice — better that than quietly using a wrong depth.

In the dialog you pick, top to bottom:

- **Input type** (asked first) — *Z-layer sub-folders* or *TIFF stacks*, as above.
- **Scope** — *Single dataset* (one dataset folder) or *Batch* (a folder holding many such datasets). A grey line under it spells out exactly what your input folder should contain for the combination you picked.
- **Input folder** and **Output folder**.
- **Projection** — *Max-Z* (keeps the **brightest** pixel at each position — the usual choice for fluorescence), *Min-Z* (keeps the **darkest**), or *Both* (runs both, into separate `max_z` and `min_z` folders).
- **Z-origin bit depth** — *16-bit* (the default; smaller and faster), *32-bit*, or *Both*. Leave it on **16-bit** unless a downstream tool specifically needs 32-bit — it's the faster choice and handles any realistic number of Z-layers.

The 8-bit "raw" preview picture is always saved — there's nothing to switch on.

What it makes — for each dataset, a `max_z_<dataset>` (or `min_z_<dataset>`) folder in your output folder; in Batch these are grouped one level down under a `max_z` / `min_z` folder. Inside:

Output	What it is
z_origin/ (16-bit) and/or **z_origin_32bit/** (32-bit)	The depth-coded projection TIFFs — point Part 2's **Z-origin TIFF folder** here. Only the bit depth you chose is written (16-bit by default).
z_layer_mapping.json	The pixel-code → depth map — this is Part 2's **Z-mapping JSON**.
raw/	A plain 8-bit projection picture for looking at (Part 2 ignores it).

> The 16-bit and 32-bit depth images hold the same information — use 16-bit unless you have more than ~65,000 Z-layers (essentially never), in which case use the 32-bit set.

> Both input types produce **exactly the same output**, so Part 2 doesn't care which one you used. Big TIFF stacks are fine too — the tool reads one slice at a time rather than loading the whole file, so a 400-slice, 700 MB time frame goes through in a couple of seconds without needing a large memory setting in Fiji.

The `z_origin` folder + `z_layer_mapping.json` this makes are exactly what Part 2 reads next.

Part 2 — 3D Z-Coordinate Extractor

Takes your **2D cell tracks** (from TrackMate or another tracker) and adds the **Z (depth) coordinate** to every detection, turning 2D tracks into 3D tracks.

It reads depth from the **Z-origin projection TIFF images** (from Part 1), where each pixel's value is a code that looks up a real depth in micrometers from the `.json` file. For each tracked point, the plugin looks at the pixel(s) under it, converts them to depth, and saves the result.

What you need before you start

File	What it is
Z-mapping JSON	A `.json` file that maps pixel codes to depths in µm, e.g. `{"0": -600.0, "1": -599.0, ...}`. (Part 1 makes this.)
Z-origin TIFF folder	A folder of TIFF images (one per time frame), 16-bit or 32-bit. These are the depth-coded projection images. (Part 1 makes this.) 8-bit and colour (RGB) TIFFs are rejected with a clear message — a pixel here is a code that has to reach into the depth map, and 8-bit only counts to 256.
Tracking CSV	Your tracks exported from TrackMate (or another tracker). Must contain X, Y, Frame, and Track ID columns.

Two things the Z-origin TIFF folder must get right. Both are checked as soon as the folder loads, so you'll see the message before the wizard goes any further:

- **Every filename must contain the frame number.** The frame index is the **last run of digits anywhere in the name**, so a prefix with its own numbers is fine — `z_origin_0007.tif` and `z_origin_32bit_0007.tif` both load as frame 7. Any zero-padding width works, and the widths may differ from file to file within the same folder — `z_7.tif`, `z_0008.tif`, and `z_00000009.tif` load as three separate frames (7, 8, 9), not as a clash. A file with **no digits at all** in its name (e.g. `frame.tif`) is rejected, and the message lists every offending file so you can rename them in one go. Note this rule is **looser than the TopoJ / direct-Z extractor's**, which needs the number at the very **end** of the name: `topoj_0007_final.tif` loads fine here as frame 7, but that same file is rejected by the TopoJ extractor. (It doesn't work the other way round — anything the TopoJ extractor accepts is fine here too, with the same frame number.)
- **Every image must be the same size as the first one.** If any TIFF's width or height differs from the first frame's, the folder is rejected and the message names the file along with both sizes. This applies to images that are **larger** as well as smaller — a larger frame used to load quietly, cropped to the first frame's top-left corner, and now stops the run instead. If you're deliberately working with a cropped subset, crop every frame to the same size first.

Opening the plugin

In Fiji, go to **Plugins > ZTracker > 3D Z-Coordinate Extractor**. The plugin runs as a simple 6-step wizard. You can move, resize, and read the Fiji **Log** window at any time while a step is open — it stays usable.

The 6 steps

1 Pick your 3 input files

Use the `...` browse buttons to select the **Z-mapping JSON**, the **Z-origin TIFF folder**, and the **tracking CSV**. Click OK.

2 Confirm the CSV format

The plugin asks how to read your CSV (header row, rows to skip, default cell radius). TrackMate exports have 3 extra info rows under the header — the defaults usually handle this. If you're unsure, leave the defaults and continue.

3 Confirm the columns

The plugin auto-detects which columns are X, Y, Frame, and Track ID and shows its guess. Check they look right and correct any that are wrong, then continue.

4 Line up the frames (offset)

Trackers often number frames starting at 0, while TIFF files often start at 1. This step lets you shift the CSV frames so they match the TIFF files. A suggested offset is already filled in (**+1 is the most common correct value**).

As you change the number, the box shows a live **per-track verdict**, and a full table is printed to the Fiji **Log** so you can check that every track lands on a real TIFF frame. When the box says all tracks map correctly, confirm.

> **Why it matters:** a wrong offset silently pairs each track point with the wrong image, giving wrong depths. Take a moment to confirm the verdict looks good.

5 Choose how to sample depth

Pick how the plugin reads pixels under each detection:

- **Sampling method:** *Radius* (average a small disk — good default), *4-Neighbor*, or *Single Pixel*. You can also choose **All** to run every method.
- **Aggregation:** *Median* (robust to odd pixels — recommended) or *Mean*. (Disabled automatically for Single Pixel, since there's only one value.)
- **Pixel convention:** leave on *Corner* (the default) unless you have a specific reason to use *Center*.

> **Not sure?** Radius + Median + Corner is a solid default for most users.

6 Choose output folder & formats

Pick where to save results and tick the formats you want (see the next section). Click OK to run.

What you get out

Format	What it's for
.npy files (`tracks_2D/` , `tracks_3D/`)	For Python analysis/plotting. 3D files hold [X, Y, Z, T].
Results Table CSV	Open in Fiji via `Analyze > Import > Results…` , or in Excel.
ROI set (.zip)	Open in Fiji's ROI Manager (`More >> Open…`) to overlay points on your image. Optional XZ/YZ side-view sets are also available.
export_report.txt	A plain-text summary of every track — how many points were kept or dropped, and why. Check this if numbers look off.

X and Y are in **pixels**, Z is in **micrometers**, and T is the frame number.

Good to know

- **No track is thrown away.** If one point is bad (e.g. its frame is missing or it falls outside the image), only that single point is dropped — the rest of the track is still exported.
- **Dropped points leave a gap** in the frame numbers rather than renumbering — this is intentional.
- **Check the Log window** after running. It reports how many points were extracted and how many were skipped, and why.
- If you picked **All** in Step 5, each method gets its own sub-folder inside your output folder.

Part 2 (alternative) — 3D Z-Extractor (TopoJ / direct-Z)

Use this instead of the standard extractor when your depth images come from **Fiji's TopoJ** (or any tool that writes the depth straight into the pixel value). Here each pixel of the projection TIFF **is** the depth in µm — there is no pixel-code and no `.json` map.

Open it via **Plugins > ZTracker > 3D Z-Extractor (TopoJ / direct-Z)**.

What's different from the standard extractor:

- **No Z-mapping JSON.** Step 1 asks for just **two** inputs — the **TopoJ Z-map TIFF folder** and your **tracking CSV**.
- **32-bit float images only.** The depth is stored as a real number per pixel, so the TIFFs must be 32-bit float. (16-bit/8-bit images are rejected with a clear message — those belong to the standard, indexed extractor.)
- **Filenames must end with the frame number.** The frame index is read from the number your filename **ends with**, so any name prefix works and the zero-padding can be any width — `frame7.tif`, `topoj_0007.tif`, and `height_map_00000100.tif` all load fine (as frames 7, 7, and 100). A file that does **not** end with a number (e.g. `topoj_0007_final.tif`) is **rejected with a message listing the offending names** — rename it so the frame number comes last, right before `.tif`.

Everything else is identical to the standard extractor: the same CSV format/column steps, the same live frame-offset check (Step 4), the same sampling/aggregation/pixel-convention choices (Step 5), the same output folder and formats (Step 6), and the same per-point drop behaviour and reports. X/Y stay in pixels, Z is in µm, T is the frame number.

> Not sure which one you have? If your projection folder came with a `z_layer_mapping.json`, use the **standard** extractor. If it's a bare folder of 32-bit float TIFFs from TopoJ with no JSON, use this **direct-Z** one.

Quick troubleshooting

Symptom	Likely fix
Lots of points show "missing frame"	Revisit **Step 4** — the frame offset is probably wrong (try +1 or 0).
Lots of points show "out of bounds"	Check your CSV X/Y match the TIFF image size, and check the pixel convention in Step 5.
Columns detected wrong	Fix them manually in **Step 3**.
Plugin not in the menu	Restart Fiji, or use `Help > Refresh Menus`.
Nothing seems to export	Make sure at least one format is ticked in **Step 6**, and read `export_report.txt`.

Symptom	Likely fix
"Inconsistent image dimensions in TIFF folder"	One TIFF is a different size from the first one in the folder — the message names it and gives both sizes. Remove it or re-crop everything to one size. Worth knowing: Part 1 doesn't compare sizes *between* timepoints, so a projection run can produce a folder like this.
"These TIFF filenames contain no frame number"	One or more TIFFs have no digits in the name, so there's no frame index to read — the message lists them all. Rename each so it carries its frame number (e.g. `z_origin_0007.tif`).
"These TIFF files resolve to the same frame number"	Two or more TIFFs point at the same frame, so one would have replaced the other — the message shows the frame number and every file that landed on it. Usually two runs mixed into one folder (`run1_0007.tif`, `run2_0007.tif` are both frame 7). Separate them into their own folders, or rename so each frame appears once.
"requires 32-bit float TIFFs" error	You opened the **TopoJ / direct-Z** extractor with indexed images — use the standard **3D Z-Coordinate Extractor** (with its JSON map) instead.
Part 1: "no Z-layer sub-folders" or "no .tif stacks found"	The **Input type** doesn't match your folder — switch it (or the Scope) and check the grey structure line under Scope.
Part 1: "slice has no readable Z in its label"	Your TIFF stack's slices aren't labelled with their depth (`z = -400.000`). Re-save the stack from Fiji with the depth labels, or use the Z-layer sub-folder layout instead.

For full technical detail, see the project README.